

ASSIGNMENT 2: SYMMETRIC CRYPTO

Due date: **Friday, Oct 2nd, 2026, 11:59 pm**
Hard deadline: **Sunday, Oct 4th, 2026, 11:59 pm**

Use Piazza for questions. Instructor and TA support ends at 2 PM on the due date. A 48-hour automatic extension is provided for upload issues. There is no penalty until the hard deadline; no submissions accepted after that date. For long-term issues, contact the instructor *before the due date*.

Total marks: 250 pts

Doing the Assignment: This assignment must be completed individually or in groups of two. You may discuss it with others, but you must not *copy* or share solutions (this includes not having a public repository with your solutions for the duration of the course).

Generative AI Disclosure. You may use generative AI for any part of this assignment and will not be penalized for doing so. However, use it as a tool rather than a substitute for understanding. If you use generative AI, include a final section titled “Generative AI Disclosure” describing how and where it was used. This section must be *between half a page and one page long*, and cannot be written by AI. Please also comment on how helpful the AI was, as the instructor is interested in learning about this.

What to submit

- The file `attacker.py`, for which you can find a template with the assignment files. Please do not include any additional non-default modules, so that we can run your code. This answers questions Q4, Q5, Q6, and Q7.
- A pdf file called `a2.pdf`, with the answers to questions Q1, Q2, and Q3, and AI disclosure (if applicable). You can create this with Microsoft Word, Google Docs, a text editor, or latex. Just make sure it is clear where each answer is, and that the pdf is readable. You can find a latex template in the assignment page.

1 Preamble

This section is general information about working on this assignment with Python. There are no questions in this section.

1.1 Strings and bytes

Before we start the crypto part of this assignment, it is good to have a solid understanding of strings and bytes in Python. You all know what a Python string is: a (usually readable) set of characters enclosed by single or double quotation marks, e.g.,

```
text='hello world!'
```

The process of converting Python strings into bytes is a type of *encoding*; converting bytes back to characters is *decoding*. There are many types of encoding formats (e.g., UTF-8, ASCII). In this course, we will use ASCII. Let's convert the text above into a bytes object:

```
text_bytes = text.encode('ascii')
```

The result is a `bytes` object. If you print this object, you will see the text again, preceded by a `b`. This is just Python interpreting the `bytes` object as a string. Also note that we can directly input the bytes as text if we declare the object as follows:

```
text_bytes=b'hello world!'
```

To convert the bytes back to text, we would use

```
text_string = text_bytes.decode('ascii')
```

1.2 Base64 encoding

Many times, we want to transfer bytes of data through a medium that is supposed to contain text data (e.g., a text file, like the ones we will use in this assignment). Base64 encoding is a technique to interpret a byte sequence as a string: the idea is to represent the bytes as bits, take groups of 6 bits, and represent each as a character. It does not matter what these bytes originally represented (text, images, video, etc.). When we encode something as Base64 characters, we will (most likely) get something “unreadable”. The Base64 characters are [A-Z], [a-z], [0,9], + and / (let's count how many characters this is: $26 + 26 + 10 +$

2 = 64 characters, which is what we can represent with 6 bits).

To convert `bytes` to Base64 and back, we can use the `base64` package. For example, let's take the bytes above (`text_bytes`) and encode them as Base64:

```
import base64
text_bytes=b'hello world!'
text_base64 = base64.b64encode(text_bytes)
print(text_base64)
```

Note that `text_base64` is another `bytes` object, but it is easy to store as text (we can convert it to text by using `.decode('ascii')`, as above). This is not meant to be read by a human, it is just a way of storing binary data in 6-bit groups (6 bits per character). When the bytes object that you are trying to encode is not a multiple of 6 (bits), you will get some “padding” at the end (this is the equal sign = that you see in this case).

The decoding process is similar:

```
text_binary = base64.b64decode(text_base64)
text = text_binary.decode('ascii')
```

Here, we decoded the Base64 string into bytes, and then decoded the bytes, interpreting them as ASCII characters.

1.3 Checking that you got this working

To check that you got this working, check that you know how to read the text (ASCII) that is encoded in this Base64 string:

```
text_base64 = 'WW91ciBkZWVvZGluZyBzZWVtcyB0byBiZSB3b3JraW5nIGNvcnJlY3RseSEh'
```

To summarize:

$$\begin{aligned} & \text{'hello!'} \xrightarrow{\text{ASCII encode}} \text{b'hello!'} \xrightarrow{\text{b64encode}} \text{b'aGVsbG8h'} \xrightarrow{\text{ASCII decode}} \text{'aGVsbG8h'} \\ & \text{'hello!'} \xleftarrow{\text{ASCII decode}} \text{b'hello!'} \xleftarrow{\text{b64decode}} \text{b'aGVsbG8h'} \xleftarrow{\text{ASCII encode}} \text{'aGVsbG8h'} \end{aligned} \quad (1)$$

1.4 XORing bytes

In this assignment, you will need to XOR bytes many times. To do so, you can use the `^` operator. To XOR `bytes` objects of arbitrary size, we can do it byte by byte:

```
A = ... # Some bytes object
B = ... # Some bytes object
xor_bytes = bytes(a ^ b for a, b in zip(A, B))
```

To check that this is working, if you take the following Base64 strings and XOR them, you should get something readable:

```
data1 = 'QHA12CEK9djY+WCq'
data2 = 'CB1BvERk1ay9gRSL'
```

If it is not working: remember to decode the Base64 representation before XORing (so that we get regular bytes).

1.5 Using AES

For this assignment, we will use the AES cryptosystem both as a block cipher (we will use ECB and CBC modes) and as a stream cipher (CTR mode). There are many ways to do this. In this assignment, we will use PyCryptodome. Install this package first.

The code below shows how to use PyCryptodome to encrypt a plaintext (`plaintext` is a `bytes` object) using a randomly-generated key (that we generated with `get_random_bytes`):

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

plaintext = b'hello, world!!!!'
key = get_random_bytes(AES.block_size)
cipher = AES.new(key, AES.MODE_ECB)
ciphertext = cipher.encrypt(plaintext)
```

Make sure you understand what this code is doing. For example, print `AES.block_size` and think if this aligns with what we have seen in the classroom.

In particular, try to encrypt a shorter plaintext, e.g., `b'hello, world!'` and see what happens. Indeed, when the input bytes are not a multiple of the block size, we cannot encrypt... unless we pad the plaintext.

Padding is simply filling up the plaintext until its length is a multiple of the block size. There are many ways to pad. One of the most popular padding mechanisms is called PKCS#7. For brevity, this document will not explain how it works, but it is very simple and Wikipedia explains it very clearly.

To pad using PyCryptodome, we do:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

plaintext = b'hello, world!'
# Pad
padded_plaintext = pad(plaintext, AES.block_size)
print(padded_plaintext)
# Encrypt
key = get_random_bytes(AES.block_size)
cipher = AES.new(key, AES.MODE_ECB)
ciphertext = cipher.encrypt(padded_plaintext)
print(ciphertext)
```

As you can see, we needed to pad the plaintext with three bytes (b'hello, world!' is 13 characters – or bytes – long). Therefore, we padded with a binary 3 (00000011), which is represented as '\x03' when printed. You can also take a look at what the ciphertext looks like when printing the bytes directly. As a human, it is hard to see how many bytes this ciphertext is, but Base64 helps us visualize bytes objects.

It might be useful for you to check what happens when you call `pad` on a plaintext that is a multiple of the block size. Is the padding ignored? Or is it padded anyway? Can you figure out why?

A few final notes:

- This pdf has only explained how to use ECB. Remember that CBC receives an IV (Initialization Vector) and CTR receives a nonce (basically, another IV).
- You can read the documentation or ask an LLM to understand how to use ECB, CBC, and CTR in PyCryptodome.
- *Important to remember:* CTR does not use padding, since we can just discard keystream bits that are not needed.

2 Detecting Modes of Operation

This is the first section of questions. These first ones are easy, but explore basic but very important concepts! Exams could ask you similar things (by giving you simple images, text, etc).

Assignment Files

- The file `lyrics-plaintext.txt` has the lyrics of a song.
- The files `lyrics-ciphertext1.txt`, `lyrics-ciphertext2.txt`, and `lyrics-ciphertext3.txt` contain encrypted versions of the song lyrics.
- The folder `ciphertext-sprites` contains twelve encrypted images.
- The file `utils.py` contains the functions that were used to load images, encrypt them, and save them into memory. It also provides a function you can use to display the images within Python.

2.1 Scenario 1: Text

In the classroom, we saw that ECB has one particular property that makes it very weak when used to encrypt more than one block. In file `lyrics-plaintext.txt`, you have the lyrics of a song. Files `lyrics-ciphertext1.txt`, `lyrics-ciphertext2.txt`, and `lyrics-ciphertext3.txt` contain an encryption of this plaintext with a random key: one is using AES-ECB, another one AES-CBC, and another one AES-CTR. The encryption process is: load the plaintext lyrics, encode as binary ASCII, encrypt with AES and a mode of operation, encode as Base64, and save as a text file.

[15 pts] Question 1: Guess which one is AES-ECB. You do not need to crack the key to make this guess (remember, the sun will die before you do). In [a2.pdf](#), explain which one is AES-ECB and how you guessed this.

[15 pts] Question 2: Guess which one is AES-CTR. Guessing which one is CTR is possible in this particular case. *Hint:* You do not need to code anything for this: just looking at the ciphertexts should be enough. All the clues to solve this are in Section 1. In [a2.pdf](#), explain which one is AES-CTR and how you guessed this.

Note: be careful not to leak the solution to these questions on Piazza, as very little information can ruin it!

2.2 Scenario 2: Images

Take a look inside the folder `ciphertext-sprites`. This folder contains 12 images: 4 have been encrypted with AES-ECB, 4 with AES-CBC, and 4 with AES-CTR. To encrypt the images, you can find some auxiliary functions inside `utils.py`. The image plaintexts are all different, and all of them have been encrypted with the same key. All images are 48×48 pixels in grayscale, so we did not use any padding (every row of the image is 48 bytes, which is exactly three AES blocks).

[30 pts] Question 3: Guess the mode of operation of each image. In `a2.pdf`, give the numbers of the images that are encrypted with each of the three modes of operation (e.g., “AES-ECB: 1, 4, 5, 6”, etc.), and explain *briefly* how you did it. *Note:* you can detect a mode of operation by ruling out the other two (there are four images with each mode of operation).

3 Attacking an Authentication Server

In the classroom, we have seen that AES is secure: we cannot brute-force it with our current computational tools. However, this does not mean one cannot attack a system that uses AES. For this section, you will work with an authentication server, written inside `server.py`.

Assignment Files

- The file `server.py` has the implementation of an authentication server (as a toy example) using AES.
- The file `tests.py` has the implementation of the tests we will run to grade your submissions.
- The file `attacker.py` is the template you must complete for this section of the assignment. There are four functions (attacks) to implement.

This is the idea:

1. A client signs up to an account by providing their name and password to the server, using the function `generate_guest_token`.
2. The server creates a token containing the client's credentials, their role, and a secret code that only the server knows. The server encrypts this token and sends it to the client, along with the nonce/IV (if applicable).
3. The client keeps the encrypted token. When they want to log in sometime in the future, they just send the token to the server, along with their name and password, using the function `read_token`.
4. To check if the log-in is correct, the server just has to decrypt the token, check that the secret code is still there, and that the name and password are correct.

This type of protocol is similar to how web servers send cookies to clients.

Take a look inside `server.py` for more details. A few things worth noticing are:

1. Every time the server encrypts with AES-CBC or AES-CTR, it generates a fresh nonce/IV, which is sent to the client.
2. When the user sends a name or a password through `generate_guest_token` and either of those contain the escape character `&`, the server returns empty bytes. This is to defend against certain attacks, and is a common sanitization/mediation technique. The server has a weaker version of the token generation, called `generate_guest_token_weak`, that does not perform this sanitization.

3. If the server encounters an error when the client calls `read_token` (e.g., incorrect padding, invalid ASCII representation, etc.) it will return the error description to the client. Although this might seem harmless, this gives additional information to the client, and this information can be used to attack the server! (see the last attack in this assignment).

The following attacks you will see are famous examples of how to attack modes of operation. There is lots of information there (including in LLMs) about how these attacks work. However, the attacks are also totally doable with just the information you have in the slides and in this pdf. You can tackle them in any order, but they are roughly sorted from easiest to hardest.

Note: for all these attacks, your script should finish within one second (usually, it takes milliseconds).

3.1 Forging an Admin Token in AES-CTR

Remember a property of stream ciphers: if we XOR anything with the ciphertext, this is equivalent to XORing with the plaintext. This is not an issue when we use proper integrity/authentication techniques. But when we don't, then bad stuff can happen.

For the next question, your goal is to call the server to generate a token (by default, this has the "guest" role), and then perform operations in the token so that, when calling `read_token`, the token passes all checks but your role has changed. My recommendation is that, to do this, you start playing with the server. For example, you can implement a function in the server that decrypts a token and prints the result as a string. Then, try generating a token, printing it, modifying something in it, and printing it again. Understand how changes in the ciphertext (encrypted token) affect the plaintext (decrypted token). Use this to your advantage to change the `role` field in the token. In all these questions, drawing the mode of operation (both encryption and decryption) can be very helpful.

Your solution should be in a function that you have inside `attacker.py` called `forge_admin_token_ctr`. You can check how we will grade this question by taking a look (and running) `tests.py`.

[50 pts] Question 4: Implement the attack `forge_admin_token_ctr` in `attacker.py`, which modifies the role in an encrypted token.

- You will get **40 points** if you change the role to `admin` (i.e., the server output from the forged token is "admin").
- You will get all **50 points** if the server prints `"super_admin"`. (This is because changing the role to a word slightly longer requires a *little* more thinking.)

When you design your attack, run `tests.py` and make sure it is working.

Note: You do not need to know what the secret key or `server_code` are to execute this attack. You must solve this the cryptographic way; Python tricks like accessing the secret key are not allowed. Your attack should finish in under a second (usually it will be around milliseconds).

3.2 Forging and Admin Token in AES-ECB

Your next task is to the same as above, but for AES-ECB. To do this, remember all the weaknesses of AES-ECB: the blocks are encrypted independently, all with the same key. There is no nonce/IV. This allows you to some reasoning at the block level. For example, let's say you have two plaintext blocks P1 and P2 and you concatenate to generate a plaintext $P=[P1, P2]$. The server encrypts this and generates $C=[C1, C2]$. You could separate the blocks, and send the server something like $C'=[C2, C1, C2]$, and the server would decrypt this into a plaintext that is $P'=[P2, P1, P2]$. You can use these manipulation techniques to generate encryptions of the plaintext blocks that you want, and craft the ciphertext block with the upgraded role.

[40 pts] Question 5: Implement the function `forge_admin_token_ecb` in `attacker.py`, which generates a valid token with the "admin" role (20pt) or "super_admin" role (40pt). Check `tests.py` to see how this will be evaluated.

3.3 Guessing the Secret Code in AES-ECB

The next attacks are a bit more complicated. Your goal now is to guess the secret code of the server, starting with AES-ECB. For this attack only, you get access to the weaker version of the token generation, which does not check if the name or password have the character `&`.

[50 pts] Question 6: Implement the function `guess_code_ecb` in `attacker.py`. Check how it will be evaluated in `tests.py`. Returning the first character of the code only gives you 50% of the grade. There is some help below in Appendix A as to how to crack the first character of the code (and you must figure out how to extend this to crack the rest).

Note: as before, you cannot somehow access the variable `code` inside the `Server` class by using Python tricks. Simply calling `generate_guest_token` with carefully-crafted `name` and `pwd` should be enough to do this question!

3.4 Guessing the Secret Code in AES-CBC

This is similar in difficulty to the previous one. You have to guess the server’s secret code when it uses AES-CBC.

To program this attack, you must exploit a “vulnerability” in the server’s implementation: when the client modifies the encrypted token and calls `read_token` on the server, the server outputs the type of exception that was raised! This opens a side-channel that allows the client to recover the code.

If you are struggling with this: don’t be afraid to look on the Web or to ask an LLM. This is a *very* famous attack that has been used in the real world.

[50 pts] Question 7: Implement the function `guess_code_cbc` in `attacker.py`, which guesses the server’s code in the encrypted token. Check `tests.py` for how this question will be evaluated. Guessing just the last character of the code will give you 50% of the grade.

A Help: cracking the first character of the code in the ECB attack

It is recommended that you play with the server before actually writing the ECB code-recovery attack in `attacker.py`. For example, you could add a new function in the server that takes a token, decrypts it, and prints it, block by block. Imagine we do this, and we test it:

```
server = Server('ECB')
name = 'simon'
pwd = 'password123'
enc_token, _ = server.generate_guest_token(name, pwd)
```

Inside the server, we can print the token (before encryption), block by block:

```
block 0: name=simon&pwd=p
block 1: assword123&role=
block 2: guest&code=VZSqa
block 3: zQvXQo6XkhC+rQo9
block 4: 2Lm4oLlwWV8
```

We can also print the encrypted token block-by-block, which gives a bunch of bytes objects:

```
block 0: b'FS\xdb\x13\xe8\x81\x04\xf2\x10"\x99\xe5*\x92\\\xa6'
block 1: b'l\t$\x17\xec\x9f\xf37>\x18\\\x84{\xd3Z\xd3'
block 2: b'\xe80~Bo\xfa*1\x90\xc7\xf1\x161\xd4\xe9\xa6'
block 3: b'\x97\xc6p\xab\xca\xc3\x81\xc0G\xf6\x03.\xa4\xff\xfb\xda'
block 4: b'`w\xcc\x8a0N\x000q\x9f\xaf\x8a\xcdw&\x9e'
```

Since ECB encrypts block-by-block *with the same key* (and there is no IV), each of the blocks of plaintext above would always be encrypted to those blocks of ciphertext below.

Now, let's try with a longer password:

```
server = Server('ECB')
name = 'simon'
pwd = 'password1234567'
enc_token, _ = server.generate_guest_token(name, pwd)
```

This prints:

```
block 0: name=simon&pwd=p
block 1: assword1234567&r
```

```
block 2: ole=guest&code=V
block 3: ZSqazQvXQo6XkhC+
block 4: rQo92Lm4oLlwWV8
```

If we print the corresponding ciphertext blocks, these are:

```
block 0: b'FS\xdb\x13\xe8\x81\x04\xf2\x10"\x99\xe5*\x92\\\xa6'
block 1: b'\x0e`\xd3\xfc\xdd\x11\xc6\x11\xac1%?\xack\x87\x04'
block 2: b'.i.M\x18\xb1\xe5\xd8GA=\x7f0\xbcu<P'
block 3: b'\xd2\x19t8"\x16\x9f\x9bd\xac\xd5o\xaa~\x96\x94'
block 4: b"r{N\x7f\xc1U\x063\xed\xec\xf2_'\xd6\x91\xeb"
```

Note that, playing as the client, we don't know what the server code is. However, we know for sure what blocks 0 and 1 of the plaintext are (because we provided the name and password), and we know *all but the last byte/character* of block 2.

Here is the trick: if we could force the server to encrypt a plaintext block that is “ole=guest&code=a”, then encrypt one that is “ole=guest&code=b”, then “ole=guest&code=c” and so on (changing the last character, and looping through all possible ASCII characters), we would learn what the corresponding ciphertexts look like! The ciphertext block of one of those plaintext blocks (in this case, the one that ends with V) would be exactly the one you see as block 2 in the code block above.

So, basically, you can brute-force one byte because there are only 2^8 possible values for this byte (since you know it's ASCII, in this case is only 2^7). This is only possible because you can input any `name` and `pwd`, and because the server is using ECB mode. However, it is one of those “tricks” that have led to real-world attacks.

Start there and, then, think how to guess the remaining characters!